

Interpolation Polynomiale Resume

Introduction

Soient $(n + 1)$ points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ d'abscisses deux à deux distinctes. **L'interpolation polynomiale** de ces points consiste à déterminer un polynôme $P \in \mathbb{R}_n[X]$ tel que

$$P(x_i) = y_i \quad \forall i \in \{0, \dots, n\}$$

Les abscisses x_i ($i \in \{0, \dots, n\}$) $\longrightarrow$ Les points d'interpolation.

Les ordonnées y_i ($i \in \{0, \dots, n\}$) $\longrightarrow$ Les valeurs d'interpolation.

Méthode d'interpolation de Lagrange

Soient $n + 1$ points de coordonnées $(x_i, y_i)_{0 \leq i \leq n}$ tels que $x_i \neq x_j, \quad \forall 0 \leq i, j \leq n, \quad i \neq j$.

Il existe un unique polynôme d'interpolation de Lagrange $P_n \in \mathbb{R}_n[X]$ vérifiant

$$P_n(x_i) = y_i, \quad \forall i \in \{0, \dots, n\}$$

Le polynôme P_n s'exprime comme suit:

$$P_n(x) = \sum_{i=0}^n y_i L_i(x), \quad x \in \mathbb{R}$$

où

$$L_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

La famille de polynômes de Lagrange $\{L_0, L_1, \dots, L_n\}$ associés aux points $(x_i, y_i), i \in \{0, \dots, n\}$, est une base de l'espace vectoriel $\mathbb{R}_n[X]$.

```
1 from numpy.polynomial import Polynomial
2
3 def base_lagrange(X, i):
4     L = Polynomial([1]) # Start with 1
5     xi = X[i, 0]
6     for j in range(len(X)):
7         if j == i:
8             continue
9         xj = X[j, 0]
10        L *= Polynomial([-xj / (xi - xj), 1 / (xi - xj)])
11    return L
12
13 def polynome_lagrange(X, Y):
14     P = Polynomial([0]) # Start with 0
15     for i in range(len(X)):
16         yi = Y[i, 0]
17         P += yi * base_lagrange(X, i)
18    return P
```

Inconvénient majeur de la méthode d'interpolation de Lagrange

- Un inconvénient majeur de la méthode d'interpolation par les polynômes de Lagrange réside dans l'ajout d'un nouveau point (x_{n+1}, y_{n+1}) à un ensemble de n points d'interpolation.
- Dans ce cas, il n'est pas numériquement simple de déduire P_{n+1} à partir de P_n . En effet, **tous les calculs doivent être refaits depuis le début.**
- Pour pallier cette difficulté, on peut utiliser **la méthode d'interpolation de Newton**, qui permet une mise à jour progressive du polynôme.

Méthode d'interpolation de Newton

Soient $n + 1$ points de coordonnées $(x_i, y_i)_{0 \leq i \leq n}$ tels que

$$x_i \neq x_j, \quad \forall i, j \in \{0, \dots, n\}, i \neq j.$$

Il existe un unique polynôme d'interpolation de Newton $P_n \in \mathbb{R}_n[X]$ vérifiant

$$P_n(x_i) = y_i, \quad \forall i \in \{0, \dots, n\}.$$

Le polynôme P_n s'exprime comme suit:

$$P_n(x) = \sum_{i=0}^n \beta_i \omega_i(x), \quad x \in \mathbb{R}$$
$$= \beta_0 \underbrace{1}_{\omega_0} + \beta_1 \underbrace{(x - x_0)}_{\omega_1} + \beta_2 \underbrace{(x - x_0)(x - x_1)}_{\omega_2} + \dots + \beta_n \underbrace{(x - x_0)(x - x_1) \dots (x - x_{n-1})}_{\omega_n}$$

où

$$\omega_i(x) = \prod_{j=0}^{i-1} (x - x_j), \quad \forall i \in \{1, \dots, n\} \quad \text{et} \quad \omega_0(x) = 1$$

```
1 from numpy.polynomial import Polynomial
2
3 def base_newton(X, i):
4     W = Polynomial([1]) # Start with 1
5     for j in range(i):
6         xj = X[j]
7         W *= Polynomial([-xj, 1]) # (x - xj)
8     return W
9
10 def polynome_newton(X, Y):
11     P = Polynomial([0]) # Start with 0
12     beta = differences_divisees(X, Y)
13     for i in range(len(X)):
14         P += beta[i] * base_newton(X, i)
15     return P
```

La famille de polynômes de Newton $\{\omega_0, \omega_1, \dots, \omega_n\}$ associés aux points (x_i, y_i) , $i \in \{0, \dots, n\}$, est une base de l'espace vectoriel $\mathbb{R}_n[X]$.

Les coefficients de Newton β_i ($i \in \{0, \dots, n\}$) peuvent être déterminés en utilisant la méthode des différences divisées, qui seront définies ci-dessous, comme suit :

$$\beta_i = [y_0, \dots, y_i].$$

Détermination des coefficients de Newton

On considère $(n + 1)$ points $(x_i, y_i)_{0 \leq i \leq n}$ tels que

$$x_i \neq x_j, \quad \forall i, j \in \{0, \dots, n\}, i \neq j$$

1. La différence divisée d'ordre 0 de x_i ($0 \leq i \leq n$) est donnée par :

$$[y_i] = y_i$$

2. La différence divisée d'ordre 1 de x_{i-1} et x_i ($0 < i \leq n$) est donnée par :

$$[y_{i-1}, y_i] = \frac{y_i - y_{i-1}}{x_i - x_{i-1}}$$

3. La différence divisée d'ordre n des $(n+1)$ points est définie par récurrence entre deux différences divisées d'ordre n comme suit:

$$[y_0, y_1, \dots, y_n] = \frac{[y_1, \dots, y_n] - [y_0, y_1, \dots, y_{n-1}]}{x_n - x_0}$$

$$\beta_i = \frac{[y_1, \dots, y_i] - [y_0, \dots, y_{i-1}]}{x_i - x_0} = [y_0, \dots, y_i] \quad : \text{une différence divisée d'ordre } i$$

```

1 import numpy as np
2
3 def differences_divisees(X, Y):
4     n = len(X)
5     table = np.zeros((n, n))
6     table[:, 0] = Y           # First column = Y values
7     for j in range(1, n): # Fill the table
8         for i in range(j, n):
9             numerator = table[i, j-1] - table[i-1, j-1]
10            denominator = X[i] - X[i-j]
11            table[i, j] = numerator / denominator
12    return np.diag(table) # Return diagonal (coefficients)

```

Avantage de la méthode de Newton

Un des avantages de la méthode de Newton pour l'interpolation des points $(x_i, y_i)_{0 \leq i \leq n}$ tels que $x_i \neq x_j, \forall i, j \in \{0, \dots, n\}, i \neq j$ est le suivant:

Si on note par P_k le polynôme d'interpolation tronqué (le polynôme de degré inférieur ou égal à k , $0 \leq k < n$, qui n'interpole que les points $(x_i, y_i)_{0 \leq i \leq k}$), exprimé dans la base des polynômes de Newton $\{\omega_0, \dots, \omega_k\}$, alors:

$$P_k(x) = \beta_0 \underbrace{1}_{\omega_0} + \beta_1 \underbrace{(x - x_0)}_{\omega_1} + \beta_2 \underbrace{(x - x_0)(x - x_1)}_{\omega_2} + \dots + \beta_k \underbrace{(x - x_0)(x - x_1) \dots (x - x_{k-1})}_{\omega_k}$$

Alors P_{k+1} , le polynôme tronqué de degré inférieur ou égal à $k+1$ interpolant les points $(x_i, y_i)_{0 \leq i \leq k+1}$, s'exprime en fonction de P_k comme suit:

$$P_{k+1}(x) = P_k(x) + \beta_{k+1} \underbrace{(x - x_0)(x - x_1) \dots (x - x_k)}_{\omega_{k+1}}$$

Par conséquent, en considérant un polynôme P_n qui interpole les $(n+1)$ points $(x_i, y_i)_{0 \leq i \leq n}$, et en ajoutant un autre point (x_{n+1}, y_{n+1}) , alors le polynôme P_{n+1} interpolant les $n+2$ points peut être déduit de P_n comme suit:

$$P_{n+1}(x) = P_n(x) + \beta_{n+1} \underbrace{(x - x_0)(x - x_1) \dots (x - x_n)}_{\omega_{n+1}}$$

Estimation de l'erreur d'interpolation

Dans le cas où les points $(x_i, y_i), i \in \{0, \dots, n\}$ sont définis à partir d'une fonction f (avec $f(x_i) = y_i$), le résultat suivant permet d'estimer l'erreur d'interpolation en un point quelconque.

Soient $x_0 < x_1 < \dots < x_n$ des nombres réels, f une fonction de classe C^{n+1} sur $[x_0, x_n]$, et P_n le polynôme d'interpolation des points $(x_i, y_i), i \in \{0, \dots, n\}$ avec $f(x_i) = y_i$.

L'erreur d'interpolation $E_n(x) = |f(x) - P_n(x)|$ en $x \in [x_0, x_n]$ vérifie:

$$E_n(x) \leq \frac{\max_{t \in [x_0, x_n]} |f^{(n+1)}(t)|}{(n+1)!} \prod_{i=0}^n |x - x_i|$$